

Smart City Traffic Intelligence Capstone

Part 2 — Python: Methodology and Findings Report

Gautam K Das

Overview

Part 2 builds a Python pipeline, from raw data to a query tool, on top of Part 1's SQL, statistics, and Power BI analysis of the Metro Interstate Traffic Volume dataset (48,204 hourly records of westbound I-94 traffic near Minneapolis–St Paul, 2012-10-02 to 2018-09-30, with weather and US federal holiday data). It comprises four independently runnable stages — data cleaning, feature engineering, visualisation, and a command line query app — plus a shared logging system with appropriate levels throughout, and a fully version controlled, incrementally committed GitHub history.

Task 1 — Data Pipeline Construction (pipeline.py)

Methodology: the raw CSV is loaded and validated against an expected schema of nine columns, then cleaned in five stages — standardising categorical text (whitespace stripping, lowercasing weather_description), parsing and validating date_time, removing duplicate rows (both exact duplicates and duplicate timestamp hours), and detecting/imputing physically impossible sensor readings using that calendar month's own median rather than a single global average. Every stage logs its own before and after counts.

Findings:

- 48,204 raw rows became 40,575 cleaned rows: 17 exact duplicates removed, and 5,430 duplicate timestamp hours (7,612 rows) collapsed to one row per hour, with zero inconsistent traffic_volume values across any duplicate group.
- 10 rows had physically impossible temp readings (≤ 0 Kelvin) and 1 row had an implausible rain_1h reading (9,831.3 mm/hour); all were imputed using their calendar month's median (e.g. January temp median 265.48K).
- weather_description was reduced from 38 to 37 unique values after case standardisation, merging variants such as 'Sky is Clear' and 'sky is clear'.

Task 2 — Feature Engineering (feature_engineering.py)

Methodology: builds features ready for machine learning on top of Task 1's cleaned data — cyclical (sine/cosine) encoding of hour alongside day of week and weekend indicators, one hot encoding of weather_main (prefix wx_) plus a derived is_severe_weather flag, z score standardisation of temp and traffic_volume, and a congestion_category derived directly from the data using its own 25th and 75th percentile thresholds rather than arbitrary fixed cutoffs.

Findings:

- Output grew from 9 to 29 columns across 40,575 rows.
- 454 of 40,575 rows (1.1%) were flagged is_severe_weather (Thunderstorm or Squall).
- Congestion split closely follows the intended quartile design: Low 10,144 rows (≤ 25 th pct), Medium 20,289 rows, High 10,142 rows (> 75 th pct) — an approximately 25/50/25 split emerging naturally from the data.

Task 3 — Visualising Traffic Patterns (visualizations.py)

Methodology: four Matplotlib charts saved to figures/, each logged on save — a bar chart of average volume by hour, a line chart contrasting weekday vs. weekend hourly averages, a histogram of the overall traffic_volume distribution, and a boxplot of traffic_volume by weather_main ordered by descending median.

Findings:

- Traffic peaks in the afternoon/evening commute (16:00–17:00, ~5,300–5,700 avg. volume), exceeding the morning peak at 07:00 (~4,770).

- Weekdays show a pronounced double peak and a substantially higher mean (3,557) than weekends, which show a flatter single midday hump and a lower mean (2,624).
- The overall traffic_volume distribution is bimodal — reflecting two distinct regimes (quiet overnight hours vs. busy daytime hours) rather than a single skewed distribution.
- Haze, Clouds, and Rain show the highest median traffic volumes; Squall is a degenerate case (a single data point in the entire dataset), consistent with Part 1's Power BI findings and its known small sample size.

Task 4 — Mini Traffic Analytics Application (mini_app/traffic_cli.py)

Methodology: a CLI built with argparse, with four subcommands, each rebuilding the cleaned, feature engineered dataset before answering: query (look up traffic and weather for a specific date and time, falling back to the nearest available hour), peak-hours (the N hours with the highest traffic), compare-weekday-weekend (weekday versus weekend summary statistics), and recommend (the hours with the lowest traffic within normal waking hours, 05:00–22:00, avoiding a trivial 3am recommendation). Every invocation logs the command and its arguments at INFO; a malformed date or time input is caught and logged as a single, clear ERROR line rather than an unhandled traceback.

Findings: the recommended low traffic travel windows are 05:00 (avg. 2,104) and 22:00 (avg. 2,224), the quiet edges of the waking day, while peak-hours correctly reproduces the 16:00–07:00 ranking found in Task 3's chart.

Task 5 — Logging, Reproducibility, and GitHub

Methodology: every module calls only logging.getLogger(__name__); no module configures its own handler. A single shared helper, logging_config.py, attaches one console handler (INFO+) and one file handler (DEBUG+) to the root logger, invoked exactly once from whichever script's entry point is actually run — satisfying 'configure a handler only in the entry point' while keeping every script independently runnable. Third party loggers (matplotlib, PIL) are explicitly raised to WARNING so the shared log, part2_python/pipeline.log, stays focused on this project's own events. All work was committed incrementally to a public GitHub repository, with a separate, descriptively named commit for each task rather than one commit at the end.

Log levels are used meaningfully throughout: DEBUG for internal values only useful for troubleshooting (e.g. scaling means/stds, quartile thresholds); INFO for expected milestones (data loaded, a stage or script completed, a figure saved, a CLI command invoked); WARNING for recoverable data issues handled automatically (duplicates removed, impossible readings imputed); and ERROR for problems that stop an operation — a full traceback for an unrecoverable pipeline failure, or a clean, single message for invalid CLI input. print() is used only for the CLI app's actual answer to a query, never for internal progress reporting.

Conclusion

Part 2 delivers a fully reproducible Python pipeline, tested at every stage on its own, that carries Part 1's cleaned data through feature engineering, visual analysis, and an interactive query tool, all under one consistent logging trail with a full audit history, and a version history built up in small, incremental commits.